

Université de Montpellier

Master 1 IA-SD
Année universitaire 2025–2026

PROJET RMI TP 1

Application distribuée Java RMI

Étudiant : Samir GHELANI

N° étudiant : 20245275

UNIVERSITÉ DE MONTPELLIER

Table des matières

1 Partie Introduction aux architecture du projet **3**

1.1 Objectif du projet . . . 3

1.2 L'architecture technique . . . 3

1.3 Motivation de l'approche . . . 3

2 Partie A **4**

2.1 A1 : Un premier objet distant : l'animal . . . 4

2.1.1 Conception et Choix Techniques . . . 4

2.1.2 Mise en œuvre et Publication . . . 4

2.1.3 Validation (Point de contrôle A) . . . 4

2.2 A2 : L'espèce : une donnée qui traverse le réseau . . . 5

2.2.1 Objectif . . . 6

2.2.2 Implémentation Techniques . . . 6

2.2.3 Analyse du comportement et validation par les tests . . . 6

2.3 A3 : Le dossier de suivi et les observateurs . . . 7

2.3.1 Partie I . . . 7

2.3.2 Partie II . . . 8

2.4 A4 : Le cabinet vétérinaire : un point d'entrée unique ! . . . 10

2.4.1 Objectif . . . 10

2.4.2 Implémentation technique . . . 11

2.4.3 Analyse du comportement et validation du controle . . . 11

2.5 A5 : Créer un patient depuis le client . . . 11

2.5.1 Objectif et démarche . . . 11

2.5.2 Implémentation . . . 11

2.5.3 Validation point de controle . . . 12

2.6 A6 : Alertes : Le serveur rappelle ses clients . . . 12

2.7 A7 : Ce qui traverse vraiment le réseau . . . 13

2.8 A8 : Application utilisable : l'interface en ligne de commande . . . 14

2.8.1 Objectif et organisation de l'interaction . . . 14

2.8.2 Flux général d'une requête et gestion des couches . . . 14

2.8.3 Traitement des saisies invalides et robustesse face aux pannes distantes 15

3 Synthèse et Regard critique **15**

3.1 Synthèse du projet . . . 15

3.2 Regard critique et améliorations possibles . . . 16

4 Usage des outils d'intelligence artificielle générative **16**
4.1 Regard critique et améliorations possibles 17

UNIVERSITÉ DE MONTPELLIER

1. Partie Introduction aux architecture du projet

1.1. Objectif du projet

Ce projet vise à concevoir et développer une application distribuée utilisant Java RMI. L'application représente une clinique vétérinaire où les données des patients (animaux) sont stockées sur un serveur JVM et peuvent être contrôlées à distance par des clients connectés au réseau.

1.2. L'architecture technique

L'application est séparée en trois repertoire indépendants afin de respecter et assurer l' isolation entre les partie de projet , les repos sont :

— **common :** contient les interfaces distantes (java.rmi.Remote) et les objets partagés . Ce dossier est partagé entre les deux autres parties . Il sert à définir les contrats du code entre les deux afin de respecter et sans exposer la logique d'implémentation .

— **serveur :** Héberge les servants (les implémentations concrètes des objets distant) , la gestion des collections de données en mémoire , ainsi que le registre RMI , c'est lui le responsable de publier les points d'entrée de l'application.

— **client :** Représente l'utilisateur , il récupère les stubs via le registre RMI et invoque les méthodes distantes .

1.3. Motivation de l'approche

Cette séparation garantit une isolation totale entre le code client et le code serveur ; le client n'a aucune connaissance de la structure et l'implémentation sur le serveur et pareil pour le serveur quand c'est le cas , il interagit et se comprend uniquement via les contrats définis dans common .

UNIVERSITÉ DE MONTPELLIER

2. Partie A

2.1. A1 : Un premier objet distant : l'animal

2.1.1. Conception et Choix Techniques

L'interface distante (ContratAnimal) : L'animal est le premier objet distant de l"application . il est défini par une interface dans le projet common qui étend java.rmi.Remote , et dont chaque méthode déclare l"exception java.rmi.RemoteExecption.

Encapsulation des données : a cette etape l interface expose uniquement des méthodes de lecture , pour consulter les attributs de base de l'animal

Le Servant : C'est l'implémentation concrète de l'interface côté serveur. Elle étend UnicastRemoteObject pour permettre son exportation et son écoute sur le réseau.

2.1.2. Mise en œuvre et Publication

Le serveur : initialise un registre RMI lui meme par (LocateRegistry.createRegistry) et publie une instance implementer de lanimal sous le nom animal1 via le port 1099 en utilisant la method rebind

coté client : le client se connecte au registre , il récupère l a reference de l"animal distant via son nom , et invoque ces méthodes .

2.1.3. Validation (Point de contrôle A)

Résultat observé : Le client distant arrive a se connecter au registre , récupérer l"objet et affiché correctement le nom d"animale .

Vérification du proxy RMI : l'execution de stub.getClas().getName() retourne bien : jdk.proxy1.$Proxy0 , ce qui prouve que le client ne manipule pas l'objet en réel sur la jvm local , mais par un proxy dynamique généré par java RMI qui intercepte et route les appels à travers le réseau

```java
public class clientMain {
    public static void main(String[] args) {
        String host = (args.length < 1) ? null : args[0];
        try {
            Registry registry = LocateRegistry.getRegistry(host, 1099);
            Contratanimal stub = (Contratanimal) registry.lookup("animal1");
            System.out.println("le nom de premier animal est "+ stub.getNom());
            System.out.println(""+ stub.getClass().getName());
        } catch (Exception e) {
            System.err.println("Client exception: " + e);
            e.printStackTrace();
        }
    }
}
```

```
Javadoc  Terminal  Coverage  Console  Git Staging
<terminated> clientMain [Java Application] C:\Program Files\Java\jdk-26.0.2.1\bin\javaw.exe (25 sept. 2026, 06:30:4
le nom de premier animal est jak
jdk.proxy1.$Proxy0
```

```java
    public static void main(String[] args) {
        try {
            Registry registre = LocateRegistry.createRegistry(1099);
            AnimalImp monAnimalImp = new AnimalImp("jak","amy", "bulldog", "chien");

            registre.rebind("animal1", monAnimalImp);
            System.out.println("server est pret !");
        } catch (Exception e) {
            System.err.println("Server exception: " + e);
            e.printStackTrace();
        }
    }
}
```

```
Javadoc  Terminal  Coverage  Console  Git Staging
serveurMain [Java Application] C:\Program Files\Java\jdk-26.0.2.1\bin\javaw.exe (25 sept. 2026, 06:28:34 elapsed 0:34:22) [pid: 12772]
server est pret !
```

FIGURE 1

```
Animall
serveurI
samirs_projetc
            System.err.println("Client exception: " + e);
            e.printStackTrace();
        }
    }

}
```

```
Javadoc  Terminal  Coverage  Console  Git Staging
<terminated> clientMain [Java Application] C:\Program Files\Java\jdk-26.0.2.1\bin\javaw.exe (25 sept. 2026,
le nom de premier animal est jak
Maître : amy
Race : bulldog
Espèce reçue : chien (Espérance de vie : 5 ans)
jdk.proxy1.$Proxy0
Nom de l'espèce après modification locale chez le client : Chien Modifié
Nom de l'espèce récupérée à nouveau depuis le serveur : chien
type f'objet :
l'animal est un objet remote ?true
l'espece est un objet remote ?false
```

FIGURE 2

2.2. A2 : L'espèce : une donnée qui traverse le réseau

2.2.1. Objectif

Le but de cette étape est de créer l'objet Espece qui fait partie de l'objet Animal manipulé à distance via proxy RMI , alors que cet objet sera transmis par valeur (copie) afin de pouvoir l'instancier localement après une sérialisation pour permettre des lectures locales performantes sans surcharger le réseau.

2.2.2. Implémentation Techniques

A. Lac classe de données sérialisable (common.Espece) : Pour qu'un objet puisse traverser le reseau d'une JVM a une autre , sa classe doit implementer l'interface java.io.Serializable . Cle autorise le moteur RMI à transformer l'objet en flux d'octets (serialisation) puis à le reconstruire en mémoire chez le client . Et sur le fichier on va implementer la class , crée les attribues et les methodes necessaires .

B. Mise à jour du contrat (common.Contratanimal) : Comme le objet Espece fera une partie de l'objet animal il faut mettre ajour l interface avec la methode pour l'implementer plutard .

C. Implementation sur serveur : le changement sur serveur.AnimalImp par lajout du attribue Espece et le mettre dans le constructeur et implementer la methode getEspece .

2.2.3. Analyse du comportement et validation par les tests

— **Préparation des tests :** Dans la classe serveurMain , le code est configuré avec la création du registre RMI et l'enregistrement d'un aimal associé a un objet Espece . Coté client , on récupere le stub de l'animal pour consulter ses attributs (nom , race , maitre) ainsi que l'objet espece transmis par valeur .

— **Vérification de la nature des objets :** l »affichage de la classe de l'animal confirme qu'il sagit d'un proxy RMI et non objet local , contrairement à l'objet espece . Un test de type de class est ce que c'est un objet local classique .

— **Test d'isolation et de transmission par valuer :** En modifiant la valeur du nom sur l'objet Esoece recue par copie , puis en effectuant une nouvelle requete aupès du serveur , on constate que la modification est strictement local et n'as impacté l'objet distant .

Les résultats obtenus sont illustrés par les captures d'écran suivantes :

```
l'animal est un objet remote ?true
l'especе est un objet remote ?false

--- TEST DU DOSSIER DE SUIVI DISTANT ---
État de santé initial : bon etat
========================================
Nouvel état de santé : En convalescence
Dernière observation ajoutée : 10h00 : A bien pris son traitement.
```

FIGURE 3

2.3. A3 : Le dossier de suivi et les observateurs

2.3.1. Partie I

1. choix de conception : Objet distant vs Objet par valeur :

Pour repondre au besoin ou plusieurs clients doivent pouvoire consulter et modifier simultanément le dossier de suivi d un animal : si un dossier se fait implementer comme un objet sérialisable , chaque client recoit une copie locale isolée en mémoire . Les modification apportés par un client ne seraient pas visibles par les autres . Alors le choix retenu c est que le dossier suivi est concu comme un objet distant via un proxy (stub) qui pointe vers une instance unique centrale sur le serveur qui est partagé entre tous les clients qui veulent faire un changement sur l'animal .

2. implémentation technique :

A. Le contrat distant (common.ContratDossierSuivi) : l'interface sur le dossier partagé definit les méthodes accessibles à distance pour lire et modifier l'etat de santé ainsi que l'histoire des observations .

B. l'mplementation sur le serveur DossiersuiviImpl , gere l'etat réel et centralisé du dossier sur le serveur par definit les attribus de cette class l etat de santé et une liste d »historique d'observation , et les methodes necessaire pour les manupiler .

C. Integration dans le modele Animal : L'interface Contratanimal et son implémenttion AnimalImp sont enrichies pour associer un dossier de suivi distant à chaque animal .

3. Analyse du comportement et validation :

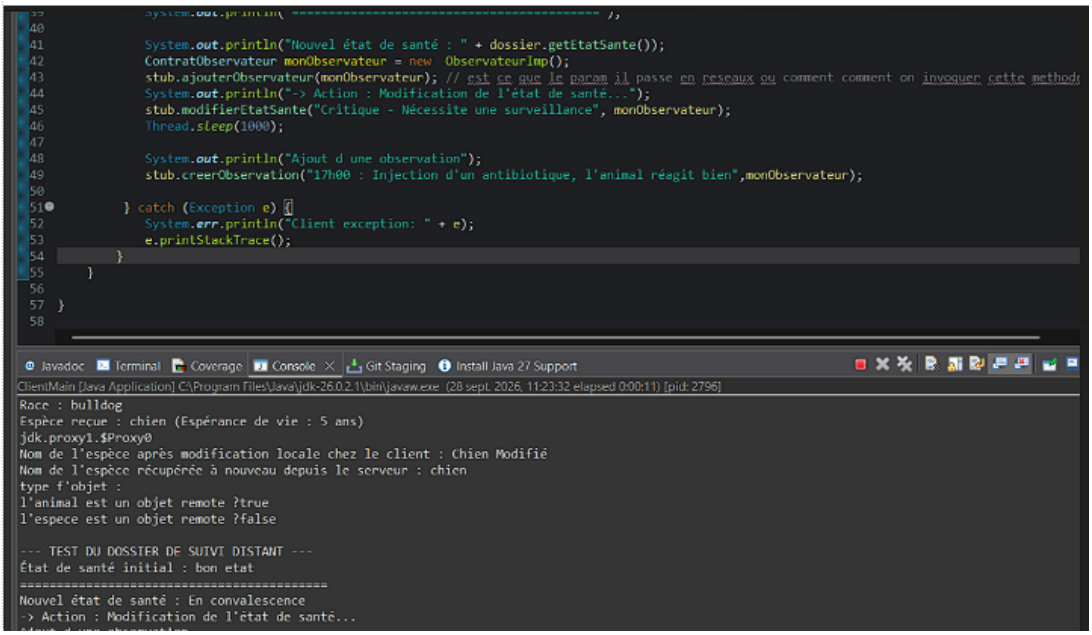


FIGURE 4

2.3.2. Partie II

Le but de cette partie est de mettre en place un système de notification en temps réel basé sur ce que on a implementer la partie passé , en utilisant le mécanisme de callback RMI , cela permet au serveur de envoyer des alertes au client qui possedent des animaux consernées .

2. Impémentation :

A. Le contrat d'observation distant (common.ContratObservateur) : Pour qu'un objet client puisse etre manipulé par le serveur , l'interface entendre java.rmi.remote et déclare les methodes avec throw RemoteException . On a definit la méthode de notification actualiser(String Message) qui sert a declancher la notification sur les client.

Voici l'extrait illustrant le contrat de l'observateur pour le mécanisme de Callback RMI :

```
public interface ContratObservateur extends Remote {
    void actualiser(String message) throws RemoteException;
}
```

B. L'implementation de cette interface c'est fait sur le client afin que le serveur puisse l'invoqué par le serveur . Cette methode etende la class UnicastRemoteObject , cette class permettre l'exportation automatique de l'objet sur le réseau et génere un stub éphémère pour que le serveur pouvoire invoquer le client apres sauvegarder sur ListObservateur .

C. Centralisation et diffusion des notification depuis le serveur : Chaque instance d animal crée sur le serveur contien une liste des client inscrit pour un animal preces , Lors une modification d'état ou observation ajouter le serveur invoque les client

concernées sauf la source de modification pour afficher cette modification via la methode actualise qui ete implementer sur le client . Depuis l »objet Observateur qui contien l'identité transmit par le client lui meme lors d'ajout sur list observateur (la connexion reseaux et l'identité de chaqu'un ete fait par le JVM implecetement .

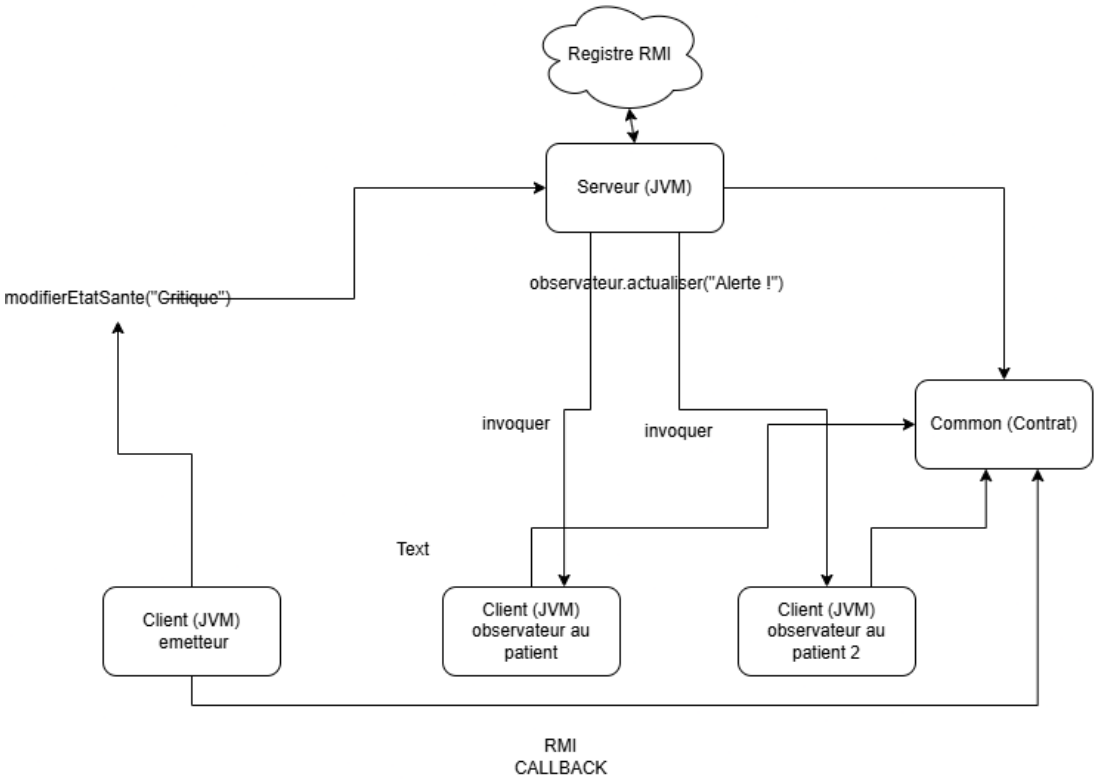


FIGURE 5 – Diagramme de séquence du mécanisme de Callback RMI (Serveur vers Client)

3. Analyse du comportement et validation par les tests :

A. Preparation des tests : Le client invoque un objet animal crée sur le serveur , instancié un objet observateur , il ajout dans la list des observateur de cet animal , on fait une modification sur l »état et l'ajout d une observation . On fait ca pour deux client apres lance deux fois le mainclient . On verra les resultat sur la console :

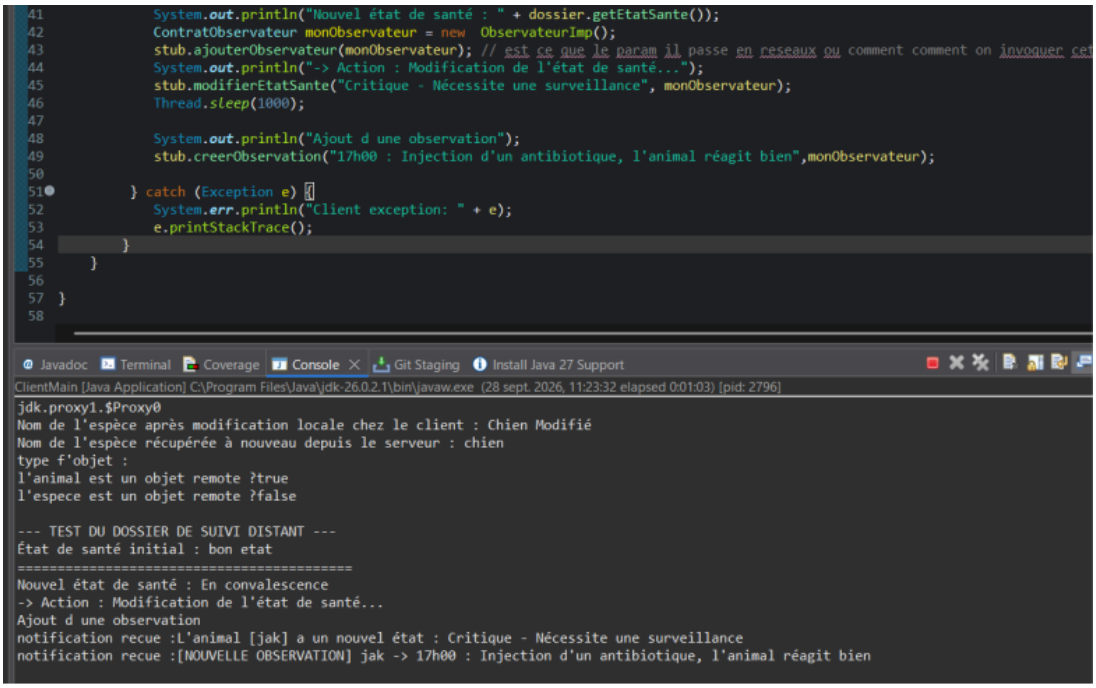


FIGURE 6

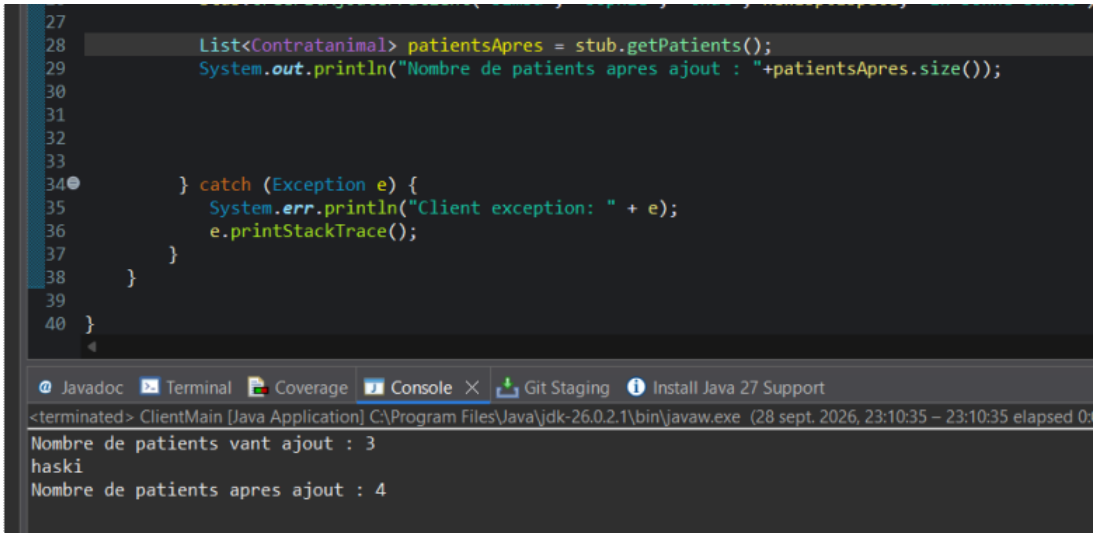


FIGURE 7

Comme c'etait attendu le serveur a executé un appel réseau inversé vers les client . Les notification s »affiche instantanément sur les client des que une modifcation effectue .

2.4. A4 : Le cabinet vétérinaire : un point d'entrée unique !

2.4.1. Objectif

Dans les étapes précédentes , chaque patient (animal) était publié individuellement dans le registre RMI . Cette approche obligrait à connaitre à l'avancele nom de chaque

animal , alors lobjectif c'avoire un cabinet qui centralise une liste d'animaux et on aura un seule objet publié dans l'annuire RMI .

2.4.2. Implémentation technique

A. Le contrat distant du cabinet : sur le repo common ; l'interface comment la majorité de notre interfaces étend java.rmi.Remote et possede les methodes necessaire pour attiendre les patients , obtnenir la liste globale avec getPatients() et la recherche par nom .

B. Conformément au choix architectural établi à l'étape A1 ou l'animal est un objet distant , le cabinet et ses méthodes seront implementer sur le serveur , alors les composant du cabinet seront des proxy réseau pointent vers les instances uniques hébergées dans la JVM du serveur et pas une copy des objets afin d »avoire une version simulaire pour tous les clients .

B. Sur le serveur main on supprimer les publication d instances animal et on a publie au lieux un seule cabinet qui declaré et enrichie avec ces animaux avant publication .

2.4.3. Analyse du comportement et validation du controle

— Le registre RMI contient q u une seule liaison cabinet qui valide la centralisation du point d'entrée

— la navigation et recherche : le cabinet contient une liste des animaux qui seront invoquée via le stub et on peut exploiter tous leurs methodes .

2.5. A5 : Créer un patient depuis le client

2.5.1. Objectif et démarche

Jusqu'à présent , les patients étaient initialisés statiquement au demarage du seveur . Pour qu'un client puisse enregistrer un nouveau patient en direct depuis son poste , on a ajouté une méthode distante sur le cabinet (creeretAjouterPatient) .

Pour respecter l'architecture d'application distribuée , l'objet animal ne doit pas etre instancie via sont console ,afin de proteger la logique du application et avoire une version pour tous les client , mais ca sera delegue au serveur qui va crée l objet et l ajouter avec les animaux existés deja .

2.5.2. Implémentation

Coté serveur (clientMain.java) : la methode sur le serveur creeretAjouterPatient sur cabinetImp , va exploter les parametre passer par le client afin de crée l'animal , le dossier suivi et lui associer au animal .

2.5.3. Validation point de controle

L'exécution du scénario de test côté client montre bien que le nombre total de patients a augmenté passant de 3 à 4 .

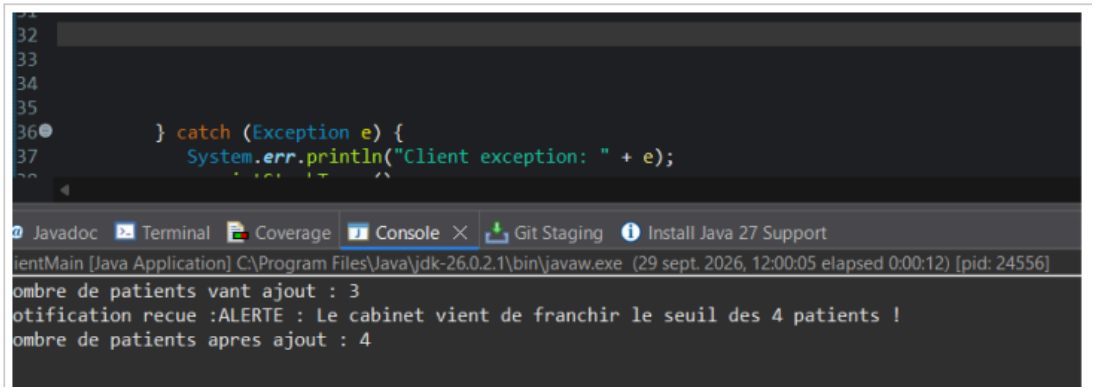


FIGURE 8

2.6. A6 : Alertes : Le serveur rappelle ses clients

Objectif : Dans la continuité du système d'observation mise en place pour le suivi des patients décrit en Partie II de A3 . Cette nouvelle étape etend le meme mécanisme de callback RMI . L'objectif est de diffuser une alerte simultanée à l »enssemble des clients lorsque un seuls précis est franchit .

A. Réutilisation et adaptation des composants d'observation : Le contratObservateur et l'implementatiion sur le client (qui gérer via le stub via UnicastRemotobject) .déja definit et validé lors de l'etape precedente , on va les reutiliser pour le meme besoin afin d'afficher notre alertes sur le console client .

B. Gestion globale des abonnements sur le serveur : Le contratObservateur pendant cette fonctionnalité il sera invoqué depuis la l'objet cabinet , ou notr but c'est superviser le nombre totale de client connecter et actif en temps réels :

— Le contrat est enrichi des méthodes abonner et desabonner pour permettre au client d'integrer et quitter la liste proprement .

— Le declanchement d'alerte est lie au nombre des client connecter , une fois le seul est franchi le serveur parcourt sa list global d'observateurs et invoque l'affichage de l'alerte sur chaque client .

C. Gestion des déconnexions inopinées : L'enjeu technique majeur de cette centralisation estt la résilence du serveur face aux pasnnes réseau ou aux fermetures brutales des connexions clients . lorsque un client quitte l'aplication sans invoquer explicitement deabonner , son stub reste enregistré dans la liste d'observateur cabinet . Pour garantir la stabilité de l'application , la boucle de notification du serveur encadre chaque appel distant (actualiser) d'un bloc de capture d'exception (try-catch). Si une RemoteException

est levée lors de la tentative de contact d'un client fantôme, le serveur l'intercepte pour ne pas interrompre le traitement, poursuit la notification des autres terminaux, et retire automatiquement la référence obsolète de sa liste d'abonné

D. Scénario de validation de la résilience : La validation de cette robustesse s'effectue en simulant un environnement multi clients dégradé. Deux clients distincts sont lancés et abonnés au cabinet. Le premier client est ensuite fermé brutalement, simulant un crash. Depuis le second client, des patients sont ajoutés jusqu'au franchissement du seuil (temporairement abaissé pour faciliter le test). Le succès du test est confirmé par deux éléments : l'affichage correct de l'alerte dans la console du second client toujours actif, et la survie du processus serveur malgré l'échec de communication avec le premier client

2.7. A7 : Ce qui traverse vraiment le réseau

A. Observation de l'echec : Lors de l'envoi d une instance de la sous-class chien (definie uniquement sur le projet client et non sur le dossier common ou serveur . L'ioperation d'ajoit de patient échoue . La trace révèle une erreur commence par java.rmi.ServerException au moment ou le client invoque la méthode distante pour ajouter le patient , et on recoit pas le succes de l'operation mais une ServerException qui indique que l'appel distant a été interrompu ca une erreur se produit sur la jvm du serveur . L'exeption encapsuler à l'interieur de la ServeurException rmi.UnmarshalExecption qui désigne l'etape de désérialisation . Cette exception informe que le serveur a bien recu l'objet sérialisé en octet via le reseau , mail il était incapable de la referencie a l un des ces classe pour la reconstruire dans sa mémoire . Et trouve plus en bas java.lang.ClassNotFoundExecption :client.chien , qui confirme notre hypothese et que le serveur n arrive pas a trouvé ce path qui contient la class (qui etait sérialisé avec le flux de données de l'objet) apres on a séparé les dossier common client serveur puisque l IDE eclipse il force la recherche et il trouve malgré que il n'a pas le droit de l avoire theoriquement comme etait expliqué au preparation enveremneent pour cette etape . La JVM lève instantanément la ClassNotFoundException, ce qui fait échouer la désérialisation, et provoque la remontée de l'erreur au terminal du client.

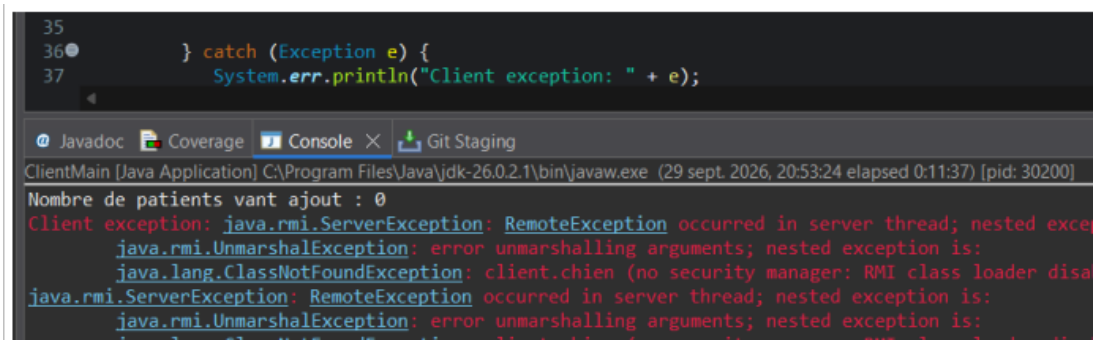


FIGURE 9

La Solution proposé : Le fichier chien.java doit etre deplacer sur common exactement sur le path de espece.java , comme ca la jvm du serveur peut l'accedé et compilé afin de pouvoire reconstruire les objets recus via le flux réseau d'apres les clients .

2.8. A8 : Application utilisable : l'interface en ligne de commande

2.8.1. Objectif et organisation de l'interaction

Pour rendre l'application pleinement utilisable par un utilisateur final sans modifier le code source, nous avons développé une interface en ligne de commande (CLI) interactive basée sur un système de menus textuels.

La conception de cette partie respecte une séparation stricte des responsabilités :

— **L'interface utilisateur (ClientMain) :** Gère l'affichage des menus, la capture des choix de l'utilisateur via un `Scanner` et la mise en forme textuelle des retours dans la console.

— **La logique client :** Fait l'intermédiaire entre le point d'acces CLI et les stubs RMI. Elle encapsule les appels réseau pour offrir des méthodes métiers claires (créer un patient, ajouter une observation, s'abonner aux alertes, etc.).

— **La couche RMI :** Assure la transparence de la communication avec le serveur distant à travers les interfaces partagées du module `common`.

2.8.2. Flux général d'une requête et gestion des couches

Le fonctionnement de l'application repose sur un parcours séquentiel rigoureux de la demande, de CLI jusqu'au serveur distant. Lorsqu'un utilisateur effectue une action (par exemple, afficher le dossier d'un patient ou modifier son état de santé), le flux s'articule ainsi :

1. **Capture et validation de la saisie (Interface CLI) :** L'utilisateur interagit avec le menu textuel. La classe principale du client capture les entrées brutes via un `Scanner`, effectue une première validation syntaxique (contrôle des types, des chaînes non vides ou du format des identifiants), et annule l'opération en cas d'erreur de saisie.
2. **Traitement par la logique client :** Une fois la saisie validée, la couche logique du client prend le relais. Elle formate les données et prépare l'appel en récupérant au préalable la référence du stub distant (fournie par le registre RMI ou par l'intermédiaire du cabinet).
3. **Transmission réseau (Invocation RMI) :** La couche RMI entre en jeu de manière transparente. Le client invoque la méthode sur le stub (ex : `cabinet.modifierEtatSante(...)`) ou transmet un objet par valeur (comme une `Espece`). Le moteur RMI sérialise les paramètres en flux d'octets et les achemine à travers le réseau TCP/IP.

4. **Traitement et centralisation (Serveur) :** Du côté de la JVM serveur, l'objet distant (le Servant) intercepte la requête, exécute la logique métier sur les données centralisées (mise à jour du dossier, ajout dans la liste), puis déclenche si nécessaire les notifications asynchrones (callbacks) vers les autres clients connectés.
5. **Retour et affichage :** Le résultat (ou l'exception en cas de panne) remonte en sens inverse à travers le réseau. Le client intercepte le retour, gère d'éventuelles `RemoteException` en cas de coupure, et affiche un compte-rendu clair dans la console de l'utilisateur.

2.8.3. Traitement des saisies invalides et robustesse face aux pannes distantes

A. Gestion des saisies utilisateur : Pour éviter les plantages de l'application lors de manipulations incorrectes (chaînes vides, identifiants inexistants, erreurs de format numérique), chaque saisie est validée en amont par des conditions de contrôle. En cas de valeur incorrecte, un message d'avertissement explicite est affiché et l'utilisateur est invité à recommencer sa saisie sans interrompre le programme.

B. Résilience face aux pannes distantes : Toutes les invocations de méthodes distantes à travers les stubs RMI sont enveloppées dans des blocs `try-catch` interceptant les exceptions de type `RemoteException`. Si le serveur s'arrête brutalement ou si la connexion réseau est coupée en cours d'exécution, le client intercepte l'erreur, prévient l'utilisateur via un message clair de rupture de service, et quitte proprement l'application au lieu de subir un arrêt brutal (crash).

3. Synthèse et Regard critique

3.1. Synthèse du projet

Le projet implémente avec succès une architecture distribuée complète en Java RMI. Les objectifs principaux ont été atteints :

— **Ce qui fonctionne :** La publication et la découverte d'objets distants via le registre RMI, la transmission d'objets par valeur (sérialisables comme `Espece`), le partage d'objets distants avec état centralisé (`DossierSuivi`), le design pattern Observer bidirectionnel (callbacks RMI pour les alertes et notifications en temps réel), ainsi que la gestion de la résilience du serveur face aux déconnexions brusques de clients.

— **Ce qui ne fonctionne pas :** Aucun dysfonctionnement bloquant n'a été relevé à l'issue des phases de test multi-clients. (partie B ET C non fourni)

— **Difficultés rencontrées :** La gestion des stubs distants dans le pattern Observer et l'identification de l'émetteur pour éviter l'auto-notification ont nécessité un soin particulier dans l'écriture des boucles de diffusion côté serveur. De même, la

configuration initiale du classpath et de la sérialisation des sous-classes (`Chien`) a requis une stricte séparation des modules (`common`).

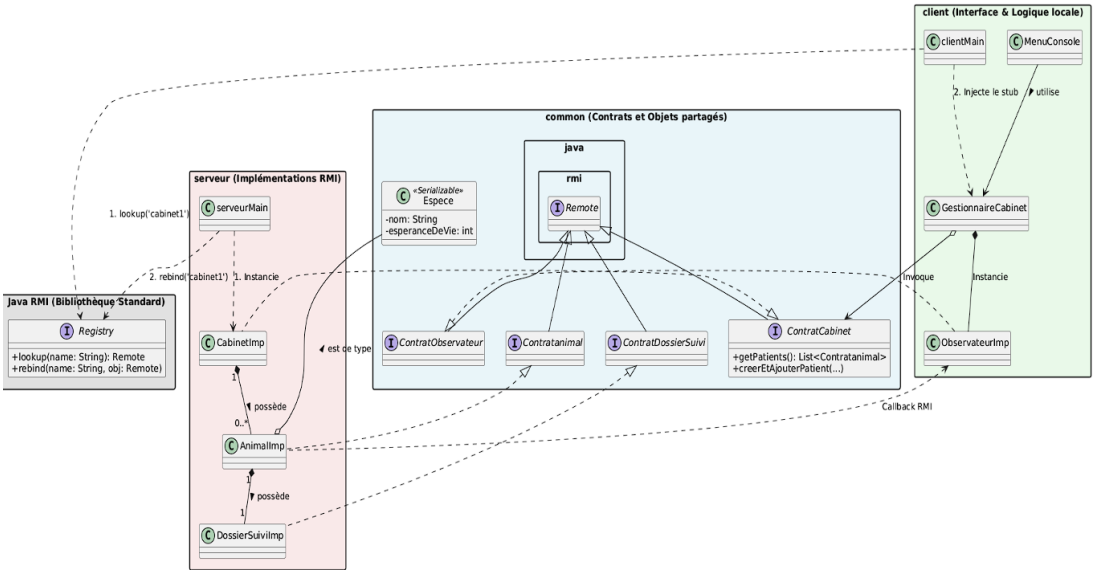


FIGURE 10 – Diagramme de classes général et architecture globale de l'application

3.2. Regard critique et améliorations possibles

Bien que fonctionnelle, l'architecture actuelle présente des axes d'amélioration notables :

— **Sécurité :** Les stubs RMI par défaut ne chiffrent pas les flux et n'authentifient pas les clients. L'intégration de protocoles de sécurité (SSL/TLS pour RMI) et d'un mécanisme d'authentification par token à la connexion serait indispensable pour un environnement de production.

— **Persistance :** Les données (patients, dossiers, observations) sont actuellement stockées en mémoire volatile sur le serveur. L'ajout d'une base de données relationnelle (via JDBC/JPA) garantirait la persistance des dossiers médicaux.

— **Ergonomie :** Remplacer l'interface en ligne de commande par une interface graphique (JavaFX) ou une API Web offrirait une meilleure expérience utilisateur.

4. Usage des outils d'intelligence artificielle générative

Des outils d'intelligence artificielle générative ont été ponctuellement utilisés au cours de ce projet :

— **Outils utilisés :** Modèles de langage basés sur l'architecture Gemini de Google.

— **Tâches confiées :** Aide à la mise en forme de la documentation LaTeX, suggestions de correction pour le débogage des exceptions RMI (notamment le comportement

du pattern Observer et des stubs distants , aides au implementation de RMI calback pour le systeme de notification),

— **Type de sollicitations :** Demandes de explications precis sur une idée avant de l'appliqué afin de gagner du temps, revues de blocs de code Java , amélioration du textes d'application console .

— **Validation et adaptation :** L'aide de l'IA a porté sur l'amélioration, l'optimisation des performances du code et l'aide au débogage, et non sur la conception de base. Les suggestions textuelles, d'optimisation et de mise en page fournies par l'IA ont été relues .

4.1. Regard critique et améliorations possibles

Bien que fonctionnelle, l'architecture actuelle présente des axes d'amélioration notables :

— **Sécurité :** Les stubs RMI par défaut ne chiffrent pas les flux et n'authentifient pas les clients. L'intégration de protocoles de sécurité (SSL/TLS pour RMI) et d'un mécanisme d'authentification par token à la connexion serait indispensable pour un environnement de production.

— **Persistance :** Les données (patients, dossiers, observations) sont actuellement stockées en mémoire volatile sur le serveur. L'ajout d'une base de données relationnelle (via JDBC/JPA) garantirait la persistance des dossiers médicaux.